SRI KRISHNADEVARAYA EDUCATIONAL TRUST

SIR M. VISVESVARAYA INSTITUTE OF TECHNOLOGY

Krishnadevarayanagar, Hunasamaranahalli, International Air Port Road, Bangalore-562 157.

(Affiliated to Visvesvaraya Technological University, Recognised by AICTE & Accredited by National Board of Accreditation, New Delhi. An ISO 9001 : 2008 Certified Institution)

Ph. : 080-2846 7248, 2847 7024/25/26 Fax : 080-2846 7081

E-mail : principal@sirmvit.edu; sirmvitbgl@gmail.com, Web : www.sirmvit.edu

DEPARTMENT OF CSE (IOT & CYBER SECURITY INCLUDING BLOCK CHIAN TECHNOLOGY)

JAVA LABORATORY

1BCS302 – 3rd SEMESTER B.E
Academic Year – 2025-26

LAB MANUAL

Prepared by:
Pooja B
Assistant Professor
Dept. of IOT

Under the Guidance of:
DR. SAVITHA CHOUDARY
Prof. and Head
Dept. of IOT

Department Vision and MISSION

VISION

Building a Proficient Internet of Things Engineers with core values to solve real time problems.

MISSION

M1: To offer state-of-the-art Internet of Things education imparting skills for building Cutting-edge and innovative applications with block chain and cyber security.

M2: To inculcate ethically and socially committed Internet of Things professionals through Value-added courses.

M3: Promoting research blend among students and enabling continuous learning

1. COURSE DESCRIPTION

Object Oriented Programming with Java introduces students to the fundamental concepts of **object-oriented programming and Java programming language**. The course enables students to develop problem-solving skills by applying programming constructs such as variables, operators, branching, looping, arrays, classes, objects, methods, inheritance, packages, interfaces, exception handling, multithreading, autoboxing, enumerations, and generics.

The laboratory component provides hands-on experience in designing, implementing, compiling, executing, testing, and debugging Java programs. Students learn to transform problem statements into object-oriented solutions and develop programs using appropriate Java features.

The course also introduces students to the use of suitable **Integrated Development Environment (IDE)** tools for Java program development and debugging.

2. COURSE OBJECTIVES

The objectives of this course are to:

1. Introduce students to the fundamentals of Java programming and object-oriented programming concepts.
2. Develop the ability to write Java programs using **variables, operators, branching, looping, arrays, and strings**.
3. Develop object-oriented solutions using **classes, objects, constructors, methods, encapsulation, and polymorphism**.
4. Enable students to implement **inheritance, packages, abstract classes, and interfaces** in Java programs.
5. Introduce **exception handling and multithreading** for developing robust and concurrent Java applications.
6. Enable students to apply advanced Java features such as **autoboxing, enumerations, and generics**.
7. Develop practical skills in using Java **IDE tools for coding, compiling, executing, testing, and debugging** programs.
8. Develop problem-solving and programming skills through hands-on laboratory exercises.

3. COURSE OUTCOMES (CO)

At the end of the course, the student will be able to:

CO	Course Outcome
CO1	Demonstrate proficiency in writing simple programs involving branching and looping structures.
CO2	Construct object-oriented solutions using classes, objects, and methods.
CO3	Develop programs using inheritance, packages, and interfaces.
CO4	Make use of exception handling and multithreading in solving complex problems.
CO5	Apply concepts of autoboxing, enumerations, and generics in program development.
CO6	Choose a suitable IDE tool to design, develop, and debug Java programs.

PO	Program Outcomes (PO)	Level	Proficiency Assessed By
PO1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.	**2**	Practical Hands-on Sessions, Lab Exercises, Viva-Voce
PO2	**Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using engineering principles.	**2**	Assignments, Practical Hands-on Sessions, Viva-Voce
PO3	**Design/development of solutions:** Design solutions for complex engineering problems and system components to meet specified needs.	**3**	Practical Hands-on Sessions, Demonstration, Mini Project
PO4	**Conduct investigations of complex problems:** Use research-based knowledge and methods to analyze, interpret, and validate engineering solutions.	**3**	Practical Hands-on Sessions, Lab Exercises, Demonstration
PO5	**Modern tool usage:** Create, select, and apply modern engineering and IT tools with an understanding of their limitations.	**3**	Practical Hands-on Sessions, Demonstration, Viva-Voce
PO6	**The engineer and society:** Apply contextual knowledge to assess societal, legal, safety, and professional responsibilities.	**2**	Demonstration, Viva-Voce
PO7	**Environment and sustainability:** Understand the impact of engineering solutions in societal and environmental contexts and the need for sustainable development.	**2**	Assignments, Case Study, Demonstration
PO8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities in engineering practice.	**2**	Viva-Voce, Assignments, Demonstration
PO9	**Individual and team work:** Function effectively as an individual and as a member or leader in multidisciplinary teams.	**2**	Team-based Practical Hands-on Sessions, Demonstration
PO10	**Communication:** Communicate effectively through technical reports, documentation, presentations, and demonstrations.	**2**	Lab Record, PPT Presentation, Demonstration
PO11	**Project management and finance:** Apply engineering and management principles to manage projects in multidisciplinary environments.	**2**	Mini Project, Practical Hands-on Sessions, Demonstration
PO12	**Life-long learning:** Recognize the need for and engage in independent and life-long learning in the context of technological change.	**2**	Self-Learning Assignment, Viva-Voce, Practical Hands-on Sessions

1. Develop a Java program to add TWO matrices of suitable order N.

```
import java.util.Scanner;

public class MatrixAddition {
        public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
      System.out.print("Enter the order of matrix: "); int n =
      sc.nextInt();

      int[][] a = new int[n][n];
      int[][] b = new int[n][n];
      int[][] sum = new int[n][n];

      System.out.println("Enter elements of first matrix:"); for (int i = 0; i < n;
      i++) {
          for (int j = 0; j < n; j++) { a[i][j] =
              sc.nextInt();
          }
      }

      System.out.println("Enter elements of second matrix:"); for (int i = 0; i <
      n; i++) {
          for (int j = 0; j < n; j++) { b[i][j] =
              sc.nextInt();
          }
      }

      for (int i = 0; i < n; i++) { for (int j = 0; j <
          n; j++) {
              sum[i][j] = a[i][j] + b[i][j];
          }
      }

      System.out.println("Sum of the two matrices:");
```

```
            for (int i = 0; i < n; i++) { for (int j = 0; j <
            n; j++) {
                        System.out.print(sum[i][j] + " ");
                  }
                  System.out.println();
            }

            sc.close();
      }
}
```

Expected Output

Enter the order of matrix: 2 Enter
elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8
Sum of the two matrices:
6 8
10 12

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a Java main method to illustrate Stack operations.

```
import java.util.Scanner;


class Stack {
    int[] stack = new int[10];
    int top = -1;


    // Push operation
    void push(int value) {
        if (top == 9) {
            System.out.println("Stack Overflow");
        } else {
            top++;
            stack[top] = value;
            System.out.println(value + " pushed into stack");
        }
    }
    // Pop operation
    void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
        } else {
            System.out.println(stack[top] + " popped from stack");
            top--;
        }
    }
    // Display operation
    void display() {
        if (top == -1) {
            System.out.println("Stack is Empty");
```

```
        } else {
            System.out.println("Stack elements are:");

            for (int i = top; i >= 0; i--) {
                System.out.println(stack[i]);
            }
        }
    }
}

class StackDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Stack s = new Stack();

        int choice, value;

        do {
            System.out.println("\n--- STACK MENU ---");
            System.out.println("1. Push");
            System.out.println("2. Pop");
            System.out.println("3. Display");
            System.out.println("4. Exit");

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:
```

```
                System.out.print("Enter value to push: ");
                value = sc.nextInt();
                s.push(value);
                break;


            case 2:
                s.pop();
                break;

            case 3:
                s.display();
                break;

            case 4:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice");
        }

    } while (choice != 4);

    sc.close();
  }
}
```

3. Create a Java class called Student with the USN, Name, Branch, Phone details as variables within it. Write a Java program to create N Student objects and print the USN, Name, Branch, and Phone of these objects with suitable headings.

```
import java.util.Scanner;

class Student { String
    USN; String Name;
    String Branch;
    String Phone;

    Student(String u, String n, String b, String p) { USN = u;
        Name = n;
        Branch = b;
        Phone = p;
    }

    void display() {
        System.out.println(USN + "\t" + Name + "\t" + Branch + "\t" + Phone
    }
}

public class StudentDemo {
                public static void main(String[] args) {

                Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
```

```
        int n = sc.nextInt();

        Student[] students = new Student[n];

        for (int i = 0; i < n; i++) {

            System.out.println("\nEnter details of student " + (i + 1));

            System.out.print("USN: "); String u =
            sc.next();

            System.out.print("Name: "); String
            name = sc.next();

            System.out.print("Branch: "); String b
            = sc.next();

            System.out.print("Phone: "); String p
            = sc.next();

            students[i] = new Student(u, name, b, p);
        }

        System.out.println("\nStudent Details");
        System.out.println("USN\tName\tBranch\tPhone");

        for (int i = 0; i < n; i++) {
            students[i].display();
        }

        sc.close();
    }
}
```

Expected Output

Enter number of students: 2
Enter details of student

USN:1MV23CS001

Name: Ravi

Branch: CSE

Phone: 9876543210

Enter details of student 2 USN:

1MV23CS002

Name: Priya Branch:

ISE Phone:

9876501234

Student Details

USN Name	Branch	Phone
1MV23CS001	Ravi	CSE 9876543210
1MV23CS002	Priya	ISE 9876501234

4. Develop a Java program to create a class named Shape. Create three sub classes, namely: circle, triangle and square. Each class has two member functions named draw and erase. Demonstrate the concepts of polymorphism by developing appropriate methods, defining member data, and implementing the main program.

```
class Shape {
    void draw() {
        System.out.println("Drawing Shape");
    }

    void erase() { System.out.println("Erasing Shape");
    }
}

class Circle extends Shape { void draw()
    {
        System.out.println("Drawing Circle");
    }

    void erase() { System.out.println("Erasing Circle");
    }
}

class Triangle extends Shape {
```

```
    void draw() {
        System.out.println("Drawing Triangle");
    }

    void erase() {
        System.out.println("Erasing Triangle");
    }
}

class Square extends Shape { void draw()
    {
        System.out.println("Drawing Square");
    }

    void erase() { System.out.println("Erasing Square");
    }
}
  public class ShapeDemo {
    public static void main(String[] args) {

        Shape s;

        s=new Circle(); s.draw();
        s.erase();

        s=new Triangle();
        s.draw();
        s.erase();

        s=new Square(); s.draw();
        s.erase();
    }
}
```

Expected Output :

Drawing Circle Erasing
Circle Drawing Triangle
Erasing Triangle
Drawing Square
Erasing Square

5. Develop a Java program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.

```
interface Resizable {
    void resizeWidth(int width); void
    resizeHeight(int height);
}

class Rectangle implements Resizable {

    int width; int
    height;

    Rectangle(int w, int h) { width = w;
        height = h;
    }

    public void resizeWidth(int w) { width =
        w;
    }

    public void resizeHeight(int h) { height = h;
    }

    void display() {
        System.out.println("Width = " + width);
        System.out.println("Height = " + height);
    }
}

public class ResizableDemo {
```

```
    public static void main(String[] args) {

        Rectangle r = new Rectangle(10, 5);

        System.out.println("Original Rectangle:"); r.display();

        r.resizeWidth(20);
        r.resizeHeight(15);

        System.out.println("\nResized Rectangle:"); r.display();
    }
}
```

Expected Output

Original Rectangle:
Width = 10
Height = 5

Resized Rectangle:
Width = 20
Height = 15

6. Develop a Java program to create a package named mypack and import & implement it in a suitable class.

Message.java

```
package mypack;


public class Message {


	public void display() {
		System.out.println("Welcome to mypack package");
	}
}
```

PackageDemo.java

```
import mypack.Message;


public class PackageDemo {
	public static void main(String[] args) {


		Message m = new Message();
		m.display();
	}
}
```

Expected Output

```
Welcome to mypack package
```

7. Develop a Java program to raise a custom exception (user-defined exception) for DivisionByZero using try, catch, throw and finally.

```
import java.util.Scanner;

class DivisionByZero extends Exception {

    DivisionByZero(String message) {
        super(message);
    }
}

public class DivisionDemo {
                public static void main(String[] args) {

                Scanner sc = new Scanner(System.in);

        System.out.print("Enter numerator: "); int a =
        sc.nextInt();

        System.out.print("Enter denominator: "); int b =
        sc.nextInt();

        try {

            if (b == 0) {
                throw new DivisionByZero("Division by zero is not allowed")

            }

            int result = a / b; System.out.println("Result = " + result);

        } catch (DivisionByZero e) {
```

```
            System.out.println("Exception: " + e.getMessage());

        } finally {

            System.out.println("Finally block executed");
        }

        sc.close();
    }
}
```

Expected Output

Enter numerator: 10
Enter denominator: 0
Exception: Division by zero is not allowed Finally block executed

8. Develop a program to create a class MyThread with a class constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

```
class MyThread extends Thread {

    MyThread() {
        super();
        System.out.println("Child thread created");
    }

    public void run() {

        for (int i = 1; i <= 5; i++) {

            System.out.println("Child Thread: " + i);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {

        MyThread t = new MyThread();

        t.start();

        for (int i = 1; i <= 5; i++) {

            System.out.println("Main Thread: " + i);
```

```
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    System.out.println(e);
                }
            }
        }
}
```

Expected Output

Child thread created Main

Thread: 1

Child Thread: 1

Main Thread: 2

Child Thread: 2

Main Thread: 3

Child Thread: 3

Main Thread: 4

Child Thread: 4

Main Thread: 5

Child Thread: 5

Note: The exact order of Main Thread and Child Thread output may change because both execute concurrently.

9. Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using valueOf().

```
import java.util.Scanner;

enum Department {
    CSE, ISE, ECE, MECH
}

public class DepartmentDemo {
                public static void main(String[] args) {

                Scanner sc = new Scanner(System.in);

        System.out.print("Enter department: "); String name =
        sc.next();

        Department department = Department.valueOf(name.toUpperCase());

        System.out.println("Selected Department: " + department);

        sc.close();
    }
}
```

Expected Output

Enter department: CSE Selected

Department: CSE

10. Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result.

```
import java.util.Scanner;

public class MarksDemo {
                public static void main(String[] args) {

                Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks of Subject 1: "); String s1 =
        sc.next();

        System.out.print("Enter marks of Subject 2: "); String s2 =
        sc.next();

        System.out.print("Enter marks of Subject 3: "); String s3 =
        sc.next();

        Integer mark1 = Integer.valueOf(s1); Integer
        mark2 = Integer.valueOf(s2); Integer mark3 =
        Integer.valueOf(s3);

        int total = mark1 + mark2 + mark3;

        double average = total / 3.0;

        System.out.println("Total = " + total);
        System.out.println("Average = " + average);

        sc.close();
    }
}
```

Expected Output

```
Enter marks of Subject 1: 80
Enter marks of Subject 2: 75
Enter marks of Subject 3: 90
Total = 245
Average = 81.66666666666667
```